RT^2 Builder Manual

1 RT^2 Input Syntax

The input system of RT^2 was developed with reference to the input formats of widely used Monte Carlo codes such as FLUKA and MCNP. Compared to these Fortran-based codes, the input format of RT^2 is relatively flexible.

An RT2 input file consists of multiple cards. In general, the order of the cards is not strictly constrained, but dependencies between them may require certain cards to appear in a specific order.

If the first non-whitespace character of a line is '#', the line is treated as a comment. Lines containing only whitespace are also ignored.

Each card follows the format described below.

```
CARD_KEY  --arg1   value1   --arg2   value2  …
```

CARD_KEY specifies the type of the card. arg1, arg2, … represent the names of the arguments associated with the card, and values for each variable can be assigned using the following strings.

Each variable must be separated by whitespace. The number of spaces and the use of tab characters are flexible and may be mixed.

The order of the arguments is not important. Some arguments may require multiple input values, in which case the values should be entered consecutively, separated by spaces.

An example is shown below.

```
CARD_KEY  --arg1    1.5   0.0   0.0  …
```

In this example, a 3-element vector {1.5, 0.0, 0.0} is assigned to the variable arg1.

Each argument associated with a card must receive a fixed number of values. If the number of provided values does not match the required length, an error may occur.

If a card becomes excessively long, the line can be continued by placing a backslash (\) at the end.

An example is shown below.

```
CARD_KEY  --arg1  value1   --arg2  value2   \
          --arg3  value3   --arg4  value4   \
          --arg5  value5   --arg6  value6
```

2 RT2builder

Radiation therapy, a primary target application for RT^2, frequently requires multiple computations within a single geometry. To optimize computational efficiency, the geometry generation code has been decoupled from the Monte Carlo core algorithm. RT2builder serves as this standalone geometry generation module, utilizing the exact same input file format as the RT2mc code. The following sections detail the cards utilized in the RT2builder code.

2.1 Program Arguments

RT2builder can be launched by executing the RT2builder command in a Windows Command Prompt or Linux terminal. The program requires certain execution arguments. You can view the list of available arguments and their descriptions by entering -h or by entering an invalid argument. The following is the output displayed when entering RT2builder -h in a Windows 10 environment:

Figure 1 List of input arguments for the RT2builder program

- input: A mandatory argument required to run the program. It specifies the name of the RT2builder input text file.
- output: Specifies the name of the log file generated during program execution. If this is not provided, the log will be printed to the terminal. This argument must be a single file name and cannot include a directory path.
- result: Specifies the directory where the log file and other generated data will be saved. If omitted, the files will be written to the current working directory where the program is executed.
- debug: Used to record internal operations—such as reading/writing data files and generating tables—into the log file. This is useful for tracing the cause of program malfunctions or unexpected crashes due to unknown errors.
- syntax: Outputs detailed descriptions of the cards used in the program.

2.2 GEO_GLOBAL

The GEO_GLOBAL card is used to control the geometry generation algorithm and file output. The available arguments are as follows:

- stepsize [float]: The step size epsilon for boundary crossing evaluator [cm]. (Default=1e-6)
- error_tolerance [float]: The geometrical error tolerance for boundary crossing evaluator. For each surface, if the area where the error occur is larger than this parameter compared to the total area, is raised. (Default=0.001)
- save_primitive [bool]: If true, all surface primitives are saved to object file format (off). (Default=false)
- save_corefined [bool] : If true, all corefined surfaces are saved to object file format (off). (Default=false)

The stepsize argument is used to determine the inside and outside regions for individual triangles

after the corefine operation. If errors occur in the generated structure due to floating-point inaccuracies, this stepsize value should be adjusted. The save_primitive and save_corefined arguments are used to save either the original surfaces generated by the input file or the results of the corefine operation, respectively, into Object File Format (.off) files. When enabled, these 3D model files are saved in the directory where the program is being executed.

2.3 Surface Cards

Surface cards generate specific shapes of closed volumes or voxel structures. RT2 constructs the entire geometry through Boolean combinations of the volumes generated by these surface cards. Conceptually, these cards are similar to a body in FLUKA, a cell in MCNP, or a solid in Geant4.

With the exception of surface cards, all other cards can be placed anywhere in the input file, in any order, as long as there are no dependency relationships. However, for surface cards, a start and end point must be declared to apply nested transformations. This is done using two specific cards: SURFACE_BEGIN and SURFACE_END. The input block for defining surfaces must follow this structure:

```
SURFACE_BEGIN
  (… surface / transformation cards …)
SURFACE_END
```

The SURFACE_BEGIN and SURFACE_END cards do not take any arguments, and there must be exactly one pair of them in the input file. Furthermore, no cards other than surface and transformation cards may be placed between them, though whitespace and comments are allowed. Any surface or transformation cards located outside of this SURFACE_BEGIN and SURFACE_END block will be ignored.

RT^2 supports a total of 11 surface cards. Their syntax, descriptions, and usage examples are provided below.

2.3.1 CUBE

The CUBE card generates a rectangular cuboid (box) surface. The available arguments are as follows:

- name [str]: The name of surface primitive. The surface name must be unique.
- center [float]: The center coordinate of this cube [cm].
- size [float]: The length of one side for each axis (xyz) [cm].

The center argument sets the coordinates for the geometric center of the cube. It strictly requires three values. Because there is no default value, this argument is mandatory. The size argument defines the lengths of the sides along the X, Y, and Z axes. The size argument requires three values, all of which must be greater than 0, and is also mandatory.

```
CUBE  --name    whatever    \
      --center  0   0   10   \
      --size    10  10  10
```

For this specific cube, the lower and upper bounds across the three axes would be x=-5 to x=5, y=-5 to y=5, and z=0 to z=10, respectively.

2.3.2 ICOSPHERE

The ICOSPHERE card generates a spherical surface using uniform triangles. The available arguments are as follows:

- name [str]: The name of surface primitive. The surface name must be unique.
- center [float]: The center coordinate of this ico-sphere [cm].
- radius [float]: The radius of the sphere [cm].
- order [int]: The order of ico-sphere tessellation algorithm. Higher order generates smoother mesh. (Default=4)

The center argument sets the center coordinates of the sphere and is a required argument. The radius argument defines the sphere's radius; this value must be greater than 0, and it is also mandatory.

The order argument dictates the number of iterations for the ico-sphere subdivision algorithm, supporting integer values from 1 to 8. The default value is 4. When the order is 1, the sphere takes the shape of a regular icosahedron. Each time the order is increased by 1, every existing triangle is subdivided into four smaller triangles, resulting in a smoother, more refined spherical surface.

The following is an example of a card that generates an ico-sphere centered at the origin with a radius of 5 cm:

```
ICOSPHERE   --name    whatever \
            --center  0  0  0  \
            --radius  5
```

Because the order argument is not explicitly provided in this example, the default value of 4 is applied, resulting in a 4th-order ico-sphere composed of 5,120 triangles.

2.3.3 UVSPHERE

The UVSPHERE card generates a sphere in the form of a solid of revolution, divided evenly along the polar and azimuthal angles relative to a central axis. The available arguments are as follows:

- name [str]: The name of surface primitive. The surface name must be unique.
- center [float]: The center coordinate of this ico-sphere [cm].
- radius [float]: The radius of the sphere [cm].
- order [int]: The order of ico-sphere tessellation algorithm. Higher order generates smoother mesh. (Default=4)

The center argument sets the center of the sphere and is a strictly required argument. The direction argument sets the axis of rotation vector used to generate the UV sphere. This argument requires three values and is automatically normalized. The radius argument sets the sphere's radius. The vertices argument takes two values, which determine the number of vertices generated at equal intervals in the polar and azimuthal directions, respectively. Both values can be set between 3 and 1000, with a default of 100. Higher values generate a sphere with a smoother surface.

The following is an example of a card that generates a UV sphere centered at the origin with a radius of 5 cm:

```
UVSPHERE    --name      whatever    \
            --center    0  0  0     \
            --radius    5
```

Because the vertices argument was not provided, the default value of 100 is applied. As a result, a sphere composed of 19,600 triangles is generated.

The shapes of the ico-sphere and UV sphere generated by the two preceding example cards are shown below:

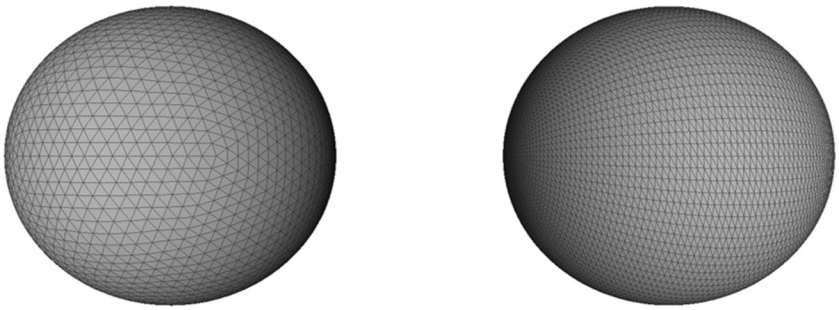

Figure 2 Sphere generated by the ICOSPHERE card (left) and UVSPHERE card (right)

Because the area of every triangle in an ico-sphere is almost uniform, it is advantageous when used as an independent region or in Boolean combinations with irregular meshes. Conversely, because a UV sphere is modeled as a solid of revolution, it is more advantageous when combined with other solids of revolution, such as cylinders or cones.

2.3.4 ELLIPSOID

The ELLIPSOID card generates an ellipsoid surface. A base sphere generated via the UV sphere method undergoes independent affine transformations along three axes to be converted into an ellipsoid. The available arguments are as follows:

- name [str]: The name of surface primitive. The surface name must be unique.
- center [float]: The center coordinate of the ellipsoidal [cm].
- direction [float]: The direction vector of the ellipsoidal. Initial axis of rotation is transformed from z-axis to this variable. (Default={0,0,1})
- radius [float]: The radius of this sphere (x, y and z-axis) [cm].
- vertices [int]: Polygon resolution of this solid. (Default={100,100}

The center argument sets the center of the ellipsoid and is a strictly required argument. The direction argument sets the axis of rotation vector used to generate the base UV sphere. This argument expects three values and is automatically normalized. The radius argument sets the radius for each axis of the ellipsoid. It requires three values, which correspond to the radii along the X, Y, and Z axes, respectively. The vertices argument takes two values, determining the number of vertices generated at equal intervals in the polar and azimuthal directions.

An ellipsoid generated using this card is identical to an ellipsoid created by applying an affine transformation card to a standard UV sphere.

2.3.5 CYLINDER

The CYLINDER card generates an n-sided prism (cylindrical) mesh. The available arguments are as follows:

- name [str]: The name of surface primitive. The surface name must be unique.
- center [float]: The center coordinate of the base of a cylinder [cm].
- radius [float]: The radius of the cylinder [cm].
- height [float]: The height vector of the cylinder [cm]. Length of the vector should be 3.
- vertices [int]: Polygon resolution of this solid. (Default=100)

The center argument sets the center coordinates of the cylinder's base. The radius argument sets the cylinder's radius. The height argument sets the height vector of the cylinder. The physical height of the cylinder is equal to the norm (magnitude) of the input vector, and the cylinder's axis aligns with this vector. The norm of the vector must be greater than 0.

2.3.6 CONE

The CONE card generates an n-sided pyramid (conical) mesh. The available arguments are as follows:

- name [str]: The name of surface primitive. The surface name must be unique.
- center [float]: The center coordinate of the base of a cylinder [cm].
- radius [float]: The radius of the cylinder [cm].
- height [float]: The height vector of the cylinder [cm]. Length of the vector should be 3.
- vertices [int]: Polygon resolution of this solid. (Default=100)

All arguments function exactly as they do in the CYLINDER card.

2.3.7 TORUS

The TORUS card generates a torus, which is a solid of revolution with a circular cross-section. The available arguments are as follows:

- name [str]: The name of surface primitive. The surface name must be unique.
- center [float]: The center coordinate of co-planar circular axis [cm].
- direction [float]: The direction vector of this solid. Initial axis rotation is transformed from z-axis to this variable (Default={0,0,1}).
- radius [float]: The radius of co-planar circle (first) and revolving circle (second).
- vertices [int]: Polygon resolution of this solid. (Default=100)

The radius argument requires two values, representing the radius of the coplanar circle and the radius of the revolving circle, respectively. The radius of the revolving circle must be strictly smaller than the radius of the coplanar circle; otherwise, an error will occur due to self-intersection.

Examples of the primitive surfaces described above can be found in [examples/basic/1_basic_solid],

and the execution results are as follows:

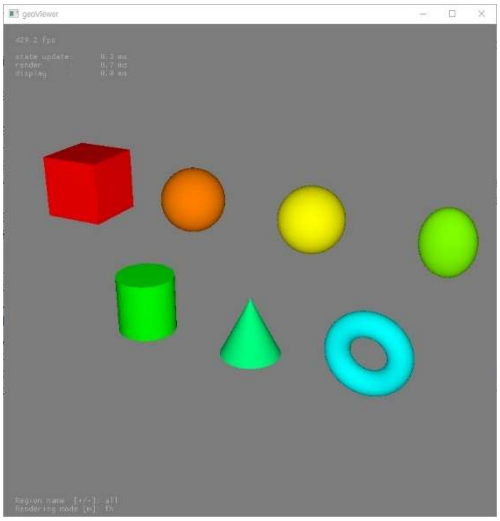

Figure 3 Examples of generated primitive surfaces

2.3.8 REVOLUTION

The REVOLUTION card generates a general solid of revolution. The available arguments are as follows:

- name [str]: The name of surface primitive. The surface name must be unique.
- entry [str]: The coordinate entry. Entry list should have form of {x1, z1, x2, z2, ... , xn, zn}. The length of entry list must even. The axis of rotation is z-axis, and the start and end points of the entry must be on the z-axis. The lines connecting each point in order must not intersect each other and must be arranged in a clockwise order.center [float]: The center coordinate of this solid [cm]. The 'entry' coordinates are the relative position of this center variable.
- direction [float]: The direction vector of this solid. Initial axis of rotation is transformed from z-axis to this variable. (Default={0,0,1})
- vertices [int]: Polygon resolution of this solid. (Default=100)

The REVOLUTION card uses the same underlying algorithm as the UVSPHERE card. However, it allows you to freely define the cross-sectional shape of the solid by specifying coordinates through the entry argument. These coordinates must consist of x and z position pairs, meaning the entry list length must be an even number. The start and end coordinates of the entry must lie on the z-axis, and the coordinates must be listed in clockwise order. Furthermore, the line segments connecting the points must not cross one another; otherwise, self-intersecting facets will be generated, which may cause an error.

Examples of generating solids of revolution using the REVOLUTION card can be found in [examples/basic/4_solid_of_revolution], and the execution results are shown below:

Figure 4 Examples of solids of revolution generated by the REVOLUTION card

2.3.9 TOROID

The TOROID card generates a general toroidal surface (toroid). The available arguments are as follows:

- name [str]: The name of surface primitive. The surface name must be unique.
- entry [str]: The coordinate entry. Entry list should have form of {x1, z1, x2, z2, ... , xn, zn}. The length of entry list must even. The axis of rotation is z-axis, and the start and end points of the entry are connected. The lines connecting each point in order must not intersect each other and must be arranged in a clockwise order.
- center [float]: The center coordinate of this solid [cm]. The 'entry' coordinates are the relative position of this center variable.
- direction [float]: The direction vector of this solid. Initial axis of rotation is transformed from z-axis to this variable. (Default={0,0,1})
- vertices [int]: Polygon resolution of this solid. (Default=100)

The usage of the TOROID card is almost identical to that of the REVOLUTION card, but the geometric conditions required for the entry coordinates are different. As before, the entry points must be listed in clockwise order. However, the start and end points of the entry must not lie on the z-axis, and none of the coordinates may cross the z-axis. The line segments connecting the entry points into a closed loop must not self-intersect.

2.3.10 MODEL

The MODEL card imports an external 3D mesh file. The available arguments are as follows:

- name [str]: The name of surface primitive. The surface name must be unique.
- file [str]: The name of 3D mesh file. Supported file extension are .off and .stl. Mesh data should not have self-intersecting facets and satisfy manifoldness.
- scale [float]: The origin mesh data is scaled by this factor. It requires three values, each a scaling factor for the x, y, and z axes. (Default={1.0,1.0,1.0})

External mesh files are supported in OFF or STL formats. They must not contain self-intersecting

facets and must be manifold. Because 3D mesh files are frequently authored using millimeters (mm) as the default unit, you can scale them to centimeters (cm) by inputting {0.1, 0.1, 0.1} into the scale argument.

An example of importing an external mesh file using the MODEL card can be found in [examples/basic/5_model], and the execution result is shown below:

Figure 5 Example of STL file input

2.3.11 VOXEL

The VOXEL card is used to input voxel data generated by auxiliary Python scripts. The available arguments are as follows:

- name [str]: The name of surface primitive. The surface name must be unique.
- file [str]: RT2 internal voxel file name. Compressed voxel file can be generated from DICOM or NIFTI file by using RT2dicom or RT2nifti.

Instructions for using the RT2dicom and RT2nifti Python scripts to generate voxel data can be found in **Sections 4.1** and **4.2** It is important to note that a voxel structure cannot be split by other surfaces. While it can be combined with other surfaces using Boolean union, attempting to split it using the Boolean difference (relative complement) operator may cause an error.

2.4 Transform Cards

Transformation cards are used to rotate, translate, and scale the coordinates of the meshes generated by surface cards. They operate based on affine transformation matrices and fully support nested transformations. The transformation logic consists of three specific cards: ROT_DEFI, which defines the affine matrix; TRANSFORM_BEGIN, which declares the start of a transformation block; and TRANSFORM_END, which defines the end of that block.

2.4.1 ROT_DEFI

The ROT_DEFI card generates the affine transformation matrix required for the transformation. The available arguments are as follows:

- name [str]: The name of rotation definition. The name must be unique.
- axis [char]: Axis of rotation. This parameter should be 'x', 'y', or 'z'. (Default=z)

- theta [float]: Rotation azimuthal angle [degree]. (Default=0.0)
- delta [float]: Translation vector [cm]. (Default={0.0,0.0,0.0})
- affine [float]: 3x4 Affine transform matrix. If this option is used, final transformation matrix is overwritten by this parameter. The transformation matrix has following structure;

$$\begin{pmatrix} x' \\ y' \\ z' \end{pmatrix} = A \begin{pmatrix} x \\ y \\ z \end{pmatrix} + t = \begin{pmatrix} A_1 & A_2 & A_3 \\ A_5 & A_6 & A_7 \\ A_9 & A_{10} & A_{11} \end{pmatrix} \begin{pmatrix} x \\ y \\ z \end{pmatrix} + \begin{pmatrix} A_4 \\ A_8 \\ A_{12} \end{pmatrix}$$

The affine argument requires a vector of exactly 12 elements, which correspond to the matrix elements from A_{1} to A_{12}. The default value for the affine argument is a vector entirely composed of zeros. If one or more elements in this vector are non-zero, the provided affine matrix is applied directly. Otherwise, the program calculates and generates the affine matrix using the axis, theta, and delta values.

2.4.2 TRANSFORM_BEGIN / TRANSFORM_END

The TRANSFORM_BEGIN and TRANSFORM_END cards apply a specific affine transformation to all surfaces encapsulated within their designated range. The available argument for the TRANSFORM_BEGIN card is:

- target [str]: The name of target transformation definition.

The target argument must exactly match the name of a transformation previously defined using a ROT_DEFI card. Any transformation block initiated by a TRANSFORM_BEGIN card must be closed by a matching TRANSFORM_END card before the overarching SURFACE_END card is declared. These two cards act like parentheses, and any surface or transformation cards placed between them will be subjected to the specified transformation. The input format is structured as follows:

```
TRANSFORM_BEGIN target=something
      (… surface / transformation cards …)
TRANSFORM_END
```

The TRANSFORM_END card does not take any arguments.

Because RT2 supports nested transformations, you can wrap surfaces in multiple TRANSFORM blocks. When evaluating these nested blocks, transformations are applied sequentially, starting with the transformation closest to the surface's definition line (working from the inside out). An example is shown below:

```
TRANSFORM_BEGIN  --target  X
    TRANSFORM_BEGIN  --target  Y
        CUBE --name    cube     \
             --center  0  0  0  \
             --size   10 10 10
    TRANSFORM_END
TRANSFORM_END
```

In this scenario, two affine transformations are applied to the cube surface. Transformation Y is applied first, followed by transformation X. The transformation matrix equation operates as follows:

$$\begin{pmatrix} x' \\ y' \\ z' \end{pmatrix} = A_X \left[A_Y \begin{pmatrix} x \\ y \\ z \end{pmatrix} + t_Y \right] + t_X$$

An example input file demonstrating nested transformations can be found in [examples/basic /2_transformation], and the rendered result is shown below:

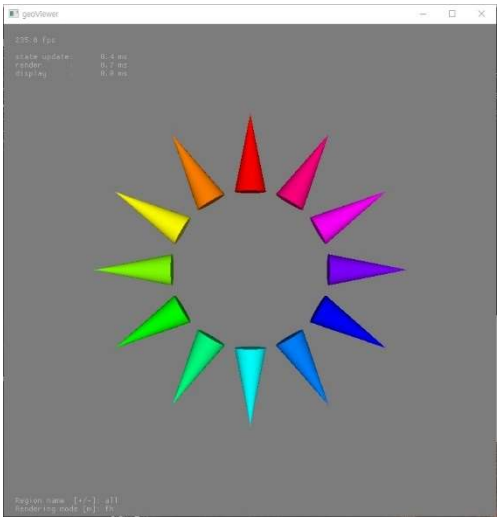

Figure 6 Example of a nested transformation

In this visual example, the origin of the entire geometry is located at the center where the cones converge, with the z-axis pointing directly toward the screen. The original base cone (red) is rotated and arrayed at 30-degree intervals in the azimuthal direction around the z-axis.

2.5 REGION

The REGION card combines your defined surfaces to create the actual spatial geometry used in the simulation. The available arguments are as follows:

- name [str]: The name of region. The region name must be unique.
- equation [str]: The list of equation element. It is interpreted as operator, operand, or decorator by following rules;

1) @ Nested region decorator. The next element is considered as region operand (if not, it is surface operand).

2) + Inside operator. The next operand indicates the interior of a surface or region (optional).

3) - Outside operator. The next operand indicates the exterior of a surface or region.

4) & And operator.

5) | Or operator.

6) (Parentheses begin operator. Paired end operator should exist. Nested parentheses are possible.

7)) Parentheses end operator. Paired begin operator should exist.

8) Else Name of surface/region operand.

The equation argument requires a Boolean geometric equation. The parser processes it correctly even if there are no spaces between operators and operands.

Precedence and Parsing Rules:

- The operational precedence of the symbols increases as you go down the list above. For example, if an outside operator (-) and a parenthesis operator appear consecutively, the parenthesis is evaluated first.
- The first seven symbols in the list act as mathematical delimiters. Any string separated by these delimiters is automatically treated as an operand name.
- By default, an operand must match the name of a surface defined by a surface card; if the surface is undefined, an error will occur.
- If an operand is prefixed with the @ decorator, it points to a previously defined region rather than a surface. The target region must be declared earlier in the input file than its current invocation.

The @ operator allows for convenient nested definitions. Here is an example of this feature:

```
REGION  --name  reg1   --equation  surf1 | surf2
REGION  --name  reg2   --equation  surf3 & -@reg1
```

Once the nested definition is parsed by the program, it executes with the exact same underlying logic as the following explicit input:

```
REGION --name reg1 --equation  surf1 | surf2
REGION --name reg2 --equation surf3 & -(surf1 | surf2)
```

2.6 Example of MCNP PRIMER

This section demonstrates how to translate the "simple example of a cylindrical storage cask" from the MCNP Primer into RT2 input syntax. The original MCNP geometry concept is shown below:

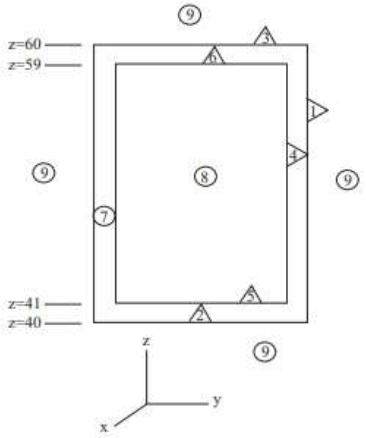

Figure 7 Geometry for a simple cylindrical cask. Numbers in triangles are surface identification numbers and numbers in circles define the cell identification number. The axis of the cask passes through the point (x = 5 cm, y = 5 cm, z = 0)

The geometry shown in **Figure 7** can be successfully defined in RT2 using the following input:

```
SURFACE_BEGIN
    CYLINDER  --name     inner_cask   \
              --center   5  5  41     \
              --height   0  0  18     \
              --radius   9
    CYLINDER  --name     outer_cask   \
              --center   5  5  40     \
              --height   0  0  20     \
              --radius  10
SURFACE_END

REGION  --name  inner_void  --equation  +inner_cask
REGION  --name  cask_shell  \
        --equation  +outer_cask & -inner_cask
REGION  --name  outer_void  --equation  -outer_cask
```

How the REGION logic works here:

1. The first REGION card defines the inner_void. It specifies this space strictly as the interior (+)

of the inner_cask surface.

2. The second REGION card utilizes the AND operator (&). Because the Outside operator (-) has higher precedence than the AND operator, the logic first identifies the exterior of inner_cask. The spatial volume that simultaneously satisfies being outside the inner_cask and inside the outer_cask becomes the cask_shell.

3. The final card defines the ambient space outside the outer_cask as the outer_void region.

The rendered result of this geometry conversion is shown below:

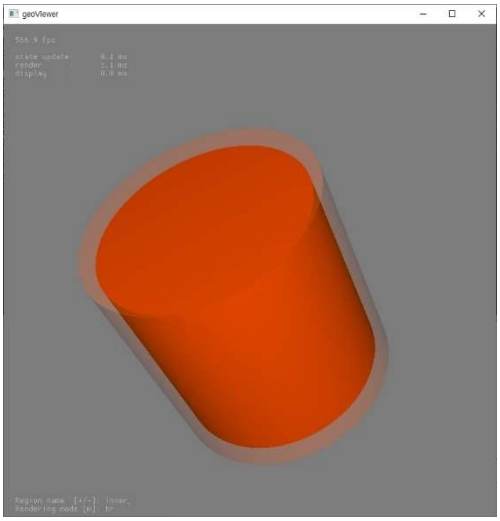

Figure 8 Rendered result of the MCNP PRIMER conversion example

3 RT2viewer

RT2viewer is a viewer program that allows you to inspect the geometry generated by the RT2builder application. You can launch RT2viewer by running the RT2viewer command in the Windows Command Prompt or a Linux terminal. Because this program relies on the GLFW library, it may not run correctly in virtual environments.

By adding -h as an execution argument, you can view a list and description of available arguments. The following is the output displayed when running RT2viewer -h in a Windows 10 environment:

```
> RT2viewer -h
Parameters: --input      | -i <filename>     Geometry file
            --resolution | -r <int2>         Renderer resolution
            --eye        | -e <float3>       Camera eye position [cm]
            --look       | -l <float3>       Camera look-at position [cm]
            --fov        | -f <float>        Camera field of view [degree]
            --help       | -h                Print this message
```

Figure 9 List of input arguments for RT2viewer

The generated geometry binary file is stored in the cache folder of your working directory. If you do

not explicitly specify a geometry file name using the --input argument, the program will default to reading the binary file located in that folder.

The initial resolution of the viewer can be set using the --resolution argument. This argument requires two integers, which represent the horizontal and vertical pixel dimensions of the viewer window, respectively. The --eye, --look, and --fov arguments are used to configure the rendering dimensions and camera setup. The structure is as follows:

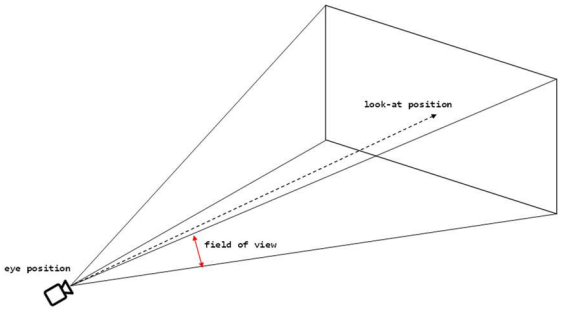

Figure 10 Camera structure of RT2viewer

Here, "field of view" refers specifically to the vertical FOV. By default, the code sets the resolution to 768x768, positions the camera at a height of 10m along the z-axis looking back toward the origin, and sets the FOV to 35 degrees.

The program utilizes the primary (first) GPU to render visualization images. Therefore, if you run the viewer while the RT2mc program is actively executing on that same primary GPU, the performance of your Monte Carlo simulation may degrade.

RT2viewer supports trackball-style camera movement:

- Left-click and drag: Fixes the "look-at" position and rotates the camera around it.
- Right-click and drag: Fixes the camera's position and rotates the "look-at" target.
- Mouse wheel: Moves the camera forward and backward along its current viewing axis.

You can change the rendering mode by pressing the 'm' key on your keyboard. The default setting is the first-hit rendering mode, which renders the boundary of the very first region the camera's rays intersect. The second rendering mode is transparent mode, which applies transparency to surfaces so you can see internal regions.

You can also cycle through target regions by pressing the '-' or '=' keys. By default, the viewer renders all regions, but using these keyboard inputs allows you to isolate and render only a specific region of interest. The following images show the results of rendering a specific region using both first-hit mode and transparent mode:

Figure 11 Examples of RT2viewer rendering results

4 Auxiliary Scripts

Auxiliary scripts are provided to easily reconfigure external data into the data formats required by RT2. Specifically, the RT2dicom and RT2nifti scripts are available to convert DICOM or NIfTI data into the voxel data format used by RT2. Once your environment variables are correctly configured, these scripts can be executed directly from the terminal.

4.1 RT2dicom

The RT2dicom script reads a set of DICOM files within a specified directory, arranges them in sequential order, and converts them into RT2-compatible voxel data. The available input arguments are as follows:

```
> RT2dicom -h
Parameters: --input   | -i  <path>       Path of DICOM CT set
            --houns   | -ho <filename>   File for hounsfield table
            --output  | -o  <filename>   File for voxel output
            --dicom   | -d               Write dicom binary for visualization
            --help    | -h               Print this message
```

Figure 12 List of input arguments for RT2dicom

The -i argument requires the directory path where the DICOM CT dataset is stored. This directory must contain a continuous sequence of DICOM images; an error will occur if any slices are missing.

The -ho argument requires a file defining the Hounsfield boundaries. The structure of this file must be exactly as follows:

$Region_1$ $Ceil_1$

$Region_2$ $Ceil_2$

…

$Region_n$ $Ceil_n$

Here, Region_n represents the name of the n-th region to be defined, and Ceil_n represents the upper limit (ceiling) of the Hounsfield Unit (HU) allocated to that n-th region. These values must be sorted in ascending order within the file. When this file is parsed, any pixel in the input DICOM dataset is assigned to Region_n if its Hounsfield value falls within the following range:

$$Ceil_{n-1} \leq HU < Ceil_n$$

An example of a file input for the -ho argument is as follows:

```
air      -300
tissue   +300
bone     +9999
```

Based on this input, all pixels with an HU value less than -300 are assigned to the air region. Pixels with an HU value of -300 or greater but less than 300 are assigned to tissue, and those from 300 up to 9999 are assigned to bone.

If all input values are valid, a file with a .vxl extension will be generated using the output name specified by the -o argument. This data can then be utilized via the VOXEL card in RT2builder.

The -d argument is used to extract the dataset in a tally data format. Because DICOM files inherently possess a 2D pixel structure, reconstructing them into 3D data can be challenging; this feature is provided to facilitate that process.

4.2 RT2nifti

The RT2nifti script converts NIfTI format files into RT2-compatible voxel data. The available input arguments are as follows:

```
>RT2nifti -h
Parameters: --input   | -i  <path>       Path of DICOM CT set
            --houns   | -ho <filename>   File for hounsfield table
            --output  | -o  <filename>   File for voxel output
            --help    | -h               Print this message
```

Figure 13 List of input arguments for RT2nifti

The execution logic and the resulting outputs are completely identical to those of the RT2dicom program.